

Task 1: Open Wireshark, capture network traffic while you refresh the Flipkart homepage, and identify one packet each belonging to the Application, Transport, Internet, and Network Access layers by examining their headers.

Capture was filtered using `ip.addr == 192.168.29.7` to isolate traffic to and from my device.

ip.addr == 192.168.29.7						
No.	Time	Source	Destination	Protocol	Length	Info
276	10.740943000	192.168.29.7	65.0.129.216	TLSv1.2	136	Application Data
277	10.741187400	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
278	10.741660600	192.168.29.7	65.0.129.216	TLSv1.2	136	Application Data
279	10.741768700	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
280	10.757899300	65.0.129.216	192.168.29.7	TCP	54	443 → 56511 [ACK] Seq=1 Ack=131 Win=4 Len=0
281	10.757900400	65.0.129.216	192.168.29.7	TLSv1.2	287	Application Data
282	10.758075000	65.0.129.216	192.168.29.7	TLSv1.2	204	Application Data
283	10.758147700	192.168.29.7	65.0.129.216	TCP	54	56511 → 443 [ACK] Seq=261 Ack=384 Win=255 Len=0
284	10.758962500	192.168.29.7	65.0.129.216	TLSv1.2	136	Application Data
285	10.759151200	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
287	10.772990000	65.0.129.216	192.168.29.7	TCP	54	443 → 56511 [ACK] Seq=384 Ack=391 Win=4 Len=0
288	10.774342100	65.0.129.216	192.168.29.7	TLSv1.2	192	Application Data
295	10.803367100	192.168.29.7	65.0.129.216	TLSv1.2	136	Application Data
296	10.803628000	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
297	10.804376300	192.168.29.7	65.0.129.216	TLSv1.2	137	Application Data
298	10.804609300	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
304	10.815059400	192.168.29.7	65.0.129.216	TLSv1.2	137	Application Data
305	10.815320000	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
308	10.821111900	65.0.129.216	192.168.29.7	TCP	54	443 → 56511 [ACK] Seq=522 Ack=652 Win=4 Len=0
309	10.821113200	65.0.129.216	192.168.29.7	TLSv1.2	398	Application Data
295	10.803367100	192.168.29.7	65.0.129.216	TLSv1.2	136	Application Data
296	10.803628000	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
297	10.804376300	192.168.29.7	65.0.129.216	TLSv1.2	137	Application Data
298	10.804609300	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
304	10.815059400	192.168.29.7	65.0.129.216	TLSv1.2	137	Application Data
305	10.815320000	192.168.29.7	65.0.129.216	TLSv1.2	102	Application Data
308	10.821111900	65.0.129.216	192.168.29.7	TCP	54	443 → 56511 [ACK] Seq=522 Ack=652 Win=4 Len=0
309	10.821113200	65.0.129.216	192.168.29.7	TLSv1.2	398	Application Data
312	10.830431300	65.0.129.216	192.168.29.7	TLSv1.2	197	Application Data
313	10.830602900	192.168.29.7	65.0.129.216	TCP	54	56511 → 443 [ACK] Seq=783 Ack=1009 Win=253 Len=0
325	10.872181600	65.0.129.216	192.168.29.7	TCP	54	443 → 56511 [ACK] Seq=1009 Ack=783 Win=4 Len=0
437	15.881665800	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.252 for any so
438	16.379116300	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 239.255.255.250 for an
441	17.878577800	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.251 for any so
640	40.619277900	192.168.29.1	192.168.29.7	ICMP	46	Echo (ping) request id=0x0000, seq=0/0, ttl=64 (repl
641	40.619484900	192.168.29.7	192.168.29.1	ICMP	46	Echo (ping) reply id=0x0000, seq=0/0, ttl=128 (req
644	42.410593900	192.168.29.7	3.110.26.57	TCP	55	57175 → 443 [ACK] Seq=1 Ack=1 Win=252 Len=1
645	42.427136200	3.110.26.57	192.168.29.7	TCP	54	443 → 57175 [ACK] Seq=1 Ack=2 Win=78 Len=0
29040	1316.1803672..	192.168.29.7	65.0.129.216	TLSv1.3	136	Application Data
29041	1316.1805732..	192.168.29.7	65.0.129.216	TLSv1.3	102	Application Data
29050	1316.1995223..	65.0.129.216	192.168.29.7	TCP	54	443 → 54518 [ACK] Seq=9995 Ack=3857 Win=8192 Len=0
29051	1316.1995238..	65.0.129.216	192.168.29.7	TLSv1.3	287	Application Data
29072	1316.2498726..	192.168.29.7	65.0.129.216	TCP	54	54518 → 443 [ACK] Seq=3857 Ack=10228 Win=65280 Len=0
29386	1328.2548848..	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.251 for any sources
29387	1328.7554302..	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.252 for any sources
29390	1330.2547313..	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 239.255.255.250 for any sources
29450	1342.6595891..	192.168.29.7	40.126.35.129	TCP	54	58164 → 443 [FIN, ACK] Seq=7383 Ack=21743 Win=65280 Len=0
29451	1342.7508226..	40.126.35.129	192.168.29.7	TCP	54	443 → 58164 [FIN, ACK] Seq=21743 Ack=7384 Win=12582656 Len=0
29452	1342.7510484..	192.168.29.7	40.126.35.129	TCP	54	58164 → 443 [ACK] Seq=7384 Ack=21744 Win=65280 Len=0
29454	1345.2434304..	192.168.29.7	34.168.184.79	TCP	55	[TCP Keep-Alive] 49682 → 7500 [ACK] Seq=1 Ack=297 Win=254 Len=1
29456	1345.4782904..	34.168.184.79	192.168.29.7	TCP	54	[TCP Keep-Alive ACK] 7500 → 49682 [ACK] Seq=297 Ack=2 Win=355 Len=0
29491	1348.2551747..	192.168.29.7	224.0.0.22	IGMPv3	54	Membership Report / Join group 224.0.0.252 for any sources
29498	1349.1087464..	192.168.29.7	3.110.26.57	TCP	66	51983 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
29499	1349.1268569..	3.110.26.57	192.168.29.7	TCP	66	443 → 51983 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=1024
29500	1349.1271441..	192.168.29.7	3.110.26.57	TCP	54	51983 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
29501	1349.1297797..	192.168.29.7	3.110.26.57	TCP	1514	51983 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 29502]
29502	1349.1297797..	192.168.29.7	3.110.26.57	TLSv1.2	440	Client Hello (SNI=careercenter.tops-int.com)
29503	1349.1470652..	3.110.26.57	192.168.29.7	TCP	54	443 → 51983 [ACK] Seq=1 Ack=1847 Win=78848 Len=0

Layer	Packet identified	Why it belongs to this layer
Application	No. 309, 65.0.129.216 -> 192.168.29.7, TLSv1.2, length 398, "Application Data"	Carries the actual encrypted webpage content sent back by the server - the real HTTP response wrapped inside TLS
Transport	No. 308, 65.0.129.216 -> 192.168.29.7, TCP, length 54, Source Port 443, Destination Port 56511,	Shows the TCP header fields: Source Port 443, Destination Port 56511,

	"443 -> 56511 [ACK] Seq=522 Ack=652 Win=4 Len=0"	sequence/acknowledgment numbers, and window size
Internet	No. 640, 192.168.29.1 -> 192.168.29.7, ICMP, length 46, "Echo (ping) request id=0x0000, seq=0/0, ttl=64"	ICMP rides directly on IP with no port numbers, so this is a clean Internet-layer-only example; the IP addresses and TTL field identify it
Network Access	Any packet (e.g. No. 295) with the "Ethernet II" (or "802.11") section expanded in the detail pane	Shows the Source/Destination MAC addresses - not visible in the packet list view, only in the expanded frame header

General identification guide, for reference:

Layer	Where to find it in Wireshark	What identifies it
Application	"Hypertext Transfer Protocol" or "Transport Layer Security" section	Contains the actual request, like GET / or the TLS handshake with Flipkart's certificate
Transport	"Transmission Control Protocol" section	Shows Source Port and Destination Port (destination port 443 for HTTPS)
Internet	"Internet Protocol Version 4" section	Shows Source and Destination IP addresses
Network Access	"Ethernet II" or "802.11" section	Shows Source and Destination MAC addresses

Task 2: Create a table comparing the OSI and TCP/IP models, listing all layers side-by-side, and for each layer, give a real-world example from any app you use.

OSI layer	TCP/IP layer	Real-world example
7. Application	Application	Composing a message in WhatsApp
6. Presentation	Application	WhatsApp encrypting your message before sending
5. Session	Application	Keeping your Whatsapp logged in
4. Transport	Transport	Splitting a large Instagram video upload into smaller segments
3. Network	Internet	Your IP address being attached so BookMyShow's server knows where to reply
2. Data Link	Network Access	Your phone's WiFi adapter framing data with its MAC address

1. Physical	Network Access	The actual WiFi radio signal or Ethernet cable carrying the bits
-------------	----------------	--

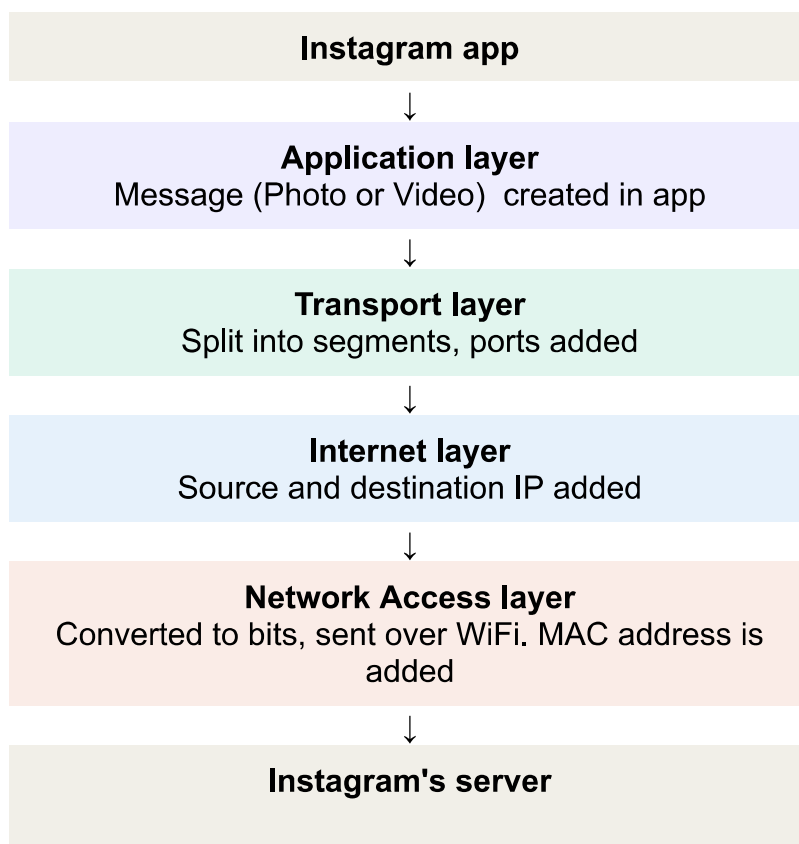
Task 3: Given a screenshot of a TCP segment header (provided by your trainer), label the fields that belong to the Transport layer and explain what each field does in 1-2 lines.

29040	1316.1803672...	192.168.29.7	65.0.129.216	TLSv1.3	136 Application Data
29041	1316.1805732...	192.168.29.7	65.0.129.216	TLSv1.3	102 Application Data
29050	1316.1995223...	65.0.129.216	192.168.29.7	TCP	54 443 → 54518 [ACK] Seq=9995 Ack=3857 Win=8192 Len=0
29051	1316.1995238...	65.0.129.216	192.168.29.7	TLSv1.3	287 Application Data
29072	1316.2498726...	192.168.29.7	65.0.129.216	TCP	54 54518 → 443 [ACK] Seq=3857 Ack=10228 Win=65280 Len=0
29386	1328.2548848...	192.168.29.7	224.0.0.22	IGMPv3	54 Membership Report / Join group 224.0.0.251 for any sources
29387	1328.7554302...	192.168.29.7	224.0.0.22	IGMPv3	54 Membership Report / Join group 224.0.0.252 for any sources
29390	1330.2547313...	192.168.29.7	224.0.0.22	IGMPv3	54 Membership Report / Join group 239.255.255.250 for any sources
29450	1342.6595891...	192.168.29.7	40.126.35.129	TCP	54 58164 → 443 [FIN, ACK] Seq=7383 Ack=21743 Win=65280 Len=0
29451	1342.7508226...	40.126.35.129	192.168.29.7	TCP	54 443 → 58164 [FIN, ACK] Seq=21743 Ack=7384 Win=12582656 Len=0
29452	1342.7510484...	192.168.29.7	40.126.35.129	TCP	54 58164 → 443 [ACK] Seq=7384 Ack=21744 Win=65280 Len=0
29454	1345.2434304...	192.168.29.7	34.168.184.79	TCP	55 [TCP Keep-Alive] 49682 → 7500 [ACK] Seq=1 Ack=297 Win=254 Len=1
29456	1345.4782904...	34.168.184.79	192.168.29.7	TCP	54 [TCP Keep-Alive ACK] 7500 → 49682 [ACK] Seq=297 Ack=2 Win=355 Len=0
29491	1348.2551747...	192.168.29.7	224.0.0.22	IGMPv3	54 Membership Report / Join group 224.0.0.252 for any sources
29498	1349.1087464...	192.168.29.7	3.110.26.57	TCP	66 51983 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
29499	1349.1268569...	3.110.26.57	192.168.29.7	TCP	66 443 → 51983 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=1024
29500	1349.1271441...	192.168.29.7	3.110.26.57	TCP	54 51983 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
29501	1349.1297797...	192.168.29.7	3.110.26.57	TCP	1514 51983 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=1460 [TCP PDU reassembled in 29502]
29502	1349.1297797...	192.168.29.7	3.110.26.57	TLSv1.2	440 Client Hello (SNI=careercenter.tops-int.com)
29503	1349.1470652...	3.110.26.57	192.168.29.7	TCP	54 443 → 51983 [ACK] Seq=1 Ack=1847 Win=78848 Len=0

- Source IP- 192.168.29.7, My Laptop IP address
- Destination IP- 3.110.26.57
- **Ports (51983 → 443):** The client randomly allotted ephemeral port 51983 to talk to the server's standard HTTPS port 443
- Sequence Number - tracks the order of bytes sent, so the receiver can reassemble segments correctly. For example, let the first byte of data by a device in a particular TCP header will have its sequence number in this field 50000. If this packet has 500 bytes of data in it, then the next packet sent by this device will have the sequence number of $50000 + 500 + 1 = 50501$.
- Acknowledgment Number - tells the sender which bytes have been successfully received so far.
- Flags (SYN, ACK, FIN) - control bits that manage connection setup, confirmation, and termination. SYN is like knocking on a door ("can I come in?"), ACK is like the person confirming they heard you ("yes, I hear you"), and FIN is like saying goodbye and closing the door properly instead of just walking away.
- Window Size - **Window Size (Win=65535)** : tells the sender how much data the receiver can accept before needing an acknowledgment.
- Checksum - used to verify the segment wasn't corrupted in transit.

- TCP Options (MSS=1460 WS=256 SACK_PERM): Extra handshake parameters. MSS=1460 means the maximum chunk of data it can receive in one packet is 1460 bytes. SACK_PERM means it supports Selective ACKs.

Task 4: Draw a diagram showing how a message sent from your device to Instagram's server passes through each TCP/IP layer, and annotate what happens at each step using simple app-related terms.



The message starts at the Application layer, where the Instagram app creates the content. The Transport layer splits it into segments and adds port numbers. The Internet layer adds source and destination IP addresses. The Network Access layer converts everything into bits and sends it out over WiFi, until it finally reaches Instagram's server.

Task 5

Pick any one packet you captured in Wireshark and write which TCP/IP layer it belongs to, how you identified it, and what real-world action in an app it represents.

Note: fill this in with your own captured packet details once Task 1 is complete. Template below.

Layer: [state which layer, e.g. Transport layer, since I am looking at the TCP header]

How identified: [e.g. I expanded the 'Transmission Control Protocol' section in Wireshark's packet details pane and saw Source Port ____ and Destination Port 443]

Real-world action: [e.g. This represents my browser establishing a connection to Flipkart's server when I refreshed the homepage, similar to how a phone opens a connection to BookMyShow's server the moment you tap on a movie listing]